

KASHTRIX

Broadcast Platform Integration & API Reference

REST + WebSocket + TCP JSONL + UDP + MCP
Playout • Channel Controller • CG Controller • CG • MAM • Ingest • NRCS/
MOS • Prompter • Traffic • As-Run • Monitoring • HA

API v1.0 / FIX44 Target Contract

17 September 2026

Document status and implementation boundary

Purpose. This is the normative integration contract for the FIX44 platform target. Every operation described here is intended to map to a real Kashtrix service/command handler, not to a decorative UI label.

Acceptance boundary. An endpoint or protocol is only production-ready after the Windows build and its corresponding hardware/network interoperability test has passed. ST 2110/PTP/NMOS/2022-7, DeckLink ANC, Dolby and multi-node HA require target infrastructure validation.

All transports feed one validated internal command/event bus. A TAKE over REST, WebSocket or TCP must execute the same authorization, validation, audit and playout handler. UDP is intentionally restricted for safety-sensitive commands unless acknowledgement/retry policy is explicitly enabled.

Contents

- Architecture
- Authentication / RBAC
- REST conventions
- WebSocket realtime API
- TCP JSONL API
- UDP API
- Playout API
- Channel Controller API
- CG Controller API
- CG Editor / Engine API
- MAM API
- Ingest API
- NRCS / MOS API
- Prompter API
- Traffic / As-Run API
- Monitoring / HA API
- MCP server
- Errors / idempotency / audit
- Complete endpoint catalogue
- Event catalogue
- Test checklist
- References

1. Architecture

Kashtrix exposes several transport adapters around the same platform services:

Transport	Default endpoint/ port	Best use	Reliability	Notes
REST/HTTPS	https:// host:9080/ api/v1	CRUD, configuration, commands, schedules, asset workflows	Reliable request/ response	Authoritative management API
WebSocket/ WSS	wss:// host:9080/ api/v1/ events/ws	Realtime state, subscriptions, bidirectional control	Ordered reliable stream	Preferred live UI/ control feed
TCP JSONL	tcp:// host:9090	Automation gateways, device systems, persistent machine clients	Ordered reliable stream	UTF-8 JSON object per LF-delimited line
UDP	udp:// host:9091	Low-latency triggers, multicast telemetry	Best effort	HMAC + timestamp/ nonce recommended; not default for critical TAKE/failover
MCP Streamable HTTP	https:// host:9080/ mcp	AI/agent tool and resource access	HTTP- native stateless core	Maps MCP tools/ resources to the same platform services

Canonical command envelope

```
{
  "v": "1.0",
  "id": "0ae2b056-94f4-4e7c-a629-9d99b3f88e5e",
  "ts": "2026-09-17T15:00:00.120Z",
  "type": "command",
  "module": "playout",
  "channelId": "KTX-PLAYOUT-01",
  "action": "take",
  "params": {
    "itemId": "7a2e4d9e-...",
    "transition": "cut"
  }
}
```

Canonical response envelope

```
{
  "v": "1.0",
  "id": "0ae2b056-94f4-4e7c-a629-9d99b3f88e5e",
  "type": "response",
  "ok": true,
  "code": "OK",
  "correlationId": "0ae2b056-94f4-4e7c-a629-9d99b3f88e5e",
  "message": "TAKE accepted",
  "data": { "channelId": "KTX-PLAYOUT-01", "onAirItemId": "7a2e4d9e-..." }
}
```

2. Authentication, permissions and audit

Interface	Authentication	Example
REST	X-Kashtrix-API-Key or Bearer JWT	X-Kashtrix-API-Key: *****
WebSocket	Bearer/API key during upgrade or first auth frame	{"type": "auth", "apiKey": "..."}
TCP	First application record must be AUTH	{"type": "auth", "apiKey": "...", "client": "automation-1"}
UDP	HMAC-SHA256 signature, timestamp and nonce; optional trusted VLAN mode	sig = HMAC(secret, timestamp + nonce + payload)
MCP	OAuth/OIDC or platform bearer credentials; server maps scopes to Kashtrix RBAC	MCP request over Streamable HTTP

Recommended roles: viewer, operator, scheduler, journalist, graphics, ingest, engineer, admin. Every mutating action writes an audit record containing identity, transport, request ID, module, target, redacted parameters, result and trace ID.

3. REST conventions

```
GET /api/v1/health
X-Kashtrix-Api-Key: YOUR_KEY
```

```
HTTP/1.1 200 OK
{
  "ok": true,
  "nodeId": "KTX-MCR-A",
  "version": "1.0",
  "modules": {
    "playout": "ready",
    "channelController": "ready",
    "cgController": "ready",
    "mam": "ready",
    "ingest": "ready",
    "nracs": "ready"
  }
}
```

Mutations accept requestId or Idempotency-Key. Reusing the same idempotency key returns the previously committed result instead of executing the operation twice.

4. Realtime WebSocket API

```
wss://mcr.example.local/api/v1/events/ws
Authorization: Bearer <token>
```

Subscribe to channel, CG and alarms

```
{
  "type": "subscribe",
  "topics": [
    "playout.KTX-PLAYOUT-01.*",
    "channelController.*",
    "cg.KTX-PLAYOUT-01.*",
    "alarm.*"
  ]
}
```

Send a realtime command on the same socket

```
{
  "v": "1.0",
  "id": "ws-take-1",
  "type": "command",
  "module": "playout",
  "channelId": "KTX-PLAYOUT-01",
  "action": "take",
  "params": {"transition": "cut"}
}
```

Realtime event

```
{
  "type": "event",
  "name": "playout.item.started",
  "sequence": 18842,
  "ts": "2026-09-17T15:02:13.040Z",
  "module": "playout",
  "channelId": "KTX-PLAYOUT-01",
  "data": {"itemId": "...", "title": "News
Bulletin", "actualStart": "2026-09-17T15:02:13.040Z"}
}
```

5. Persistent TCP JSONL API

UTF-8 newline-delimited JSON. Each complete JSON object ends in LF. Authentication must complete before commands or subscriptions.

```
{"type": "auth", "apiKey": "YOUR_KEY", "client": "automation-1"}
{"type": "subscribe", "topics": ["playout.KTX-PLAYOUT-01.*", "alarm.*"]}
{"v": "1.0", "id": "tcp-1", "type": "command", "module": "playout", "channelId": "KTX-PLAYOUT-01", "action": "next", "params": {}}
```

Required behaviors: PING/PONG heartbeat, idle timeout, message-size limit, malformed-frame rejection, correlation IDs, reconnect-safe idempotency and explicit server error records.

6. UDP API

```
{
  "v": "1.0",
  "id": "udp-cg-1",
  "ts": "2026-09-17T15:03:00.120Z",
  "nonce": "e783b4...",
  "type": "command",
  "module": "cg",
  "channelId": "KTX-PLAYOUT-01",
  "action": "play",
  "params": {"template": "LOWER_THIRD", "data": {"name": "Presenter"}},
  "signature": "hmac-sha256:..."
}
```

Safety: UDP may be lost, duplicated or reordered. Critical TAKE, failover, archive deletion, output-arm or destructive operations must use a reliable transport, or a documented UDP acknowledgement/retry policy.

7. Playout API

Method	Endpoint	Purpose
GET	/api/v1/playout/channels	List channels
GET	/api/v1/playout/channels/{channelId}/status	Program/Preview/Next/timing/output status
GET	/api/v1/playout/channels/{channelId}/playlist	Current rundown
PUT	/api/v1/playout/channels/{channelId}/playlist	Replace validated playlist
POST	/api/v1/playout/channels/{channelId}/commands	TAKE/PLAY/PAUSE/STOP/NEXT/CUE/HOLD/JIP/EXTEND/SHORTEN
POST	/api/v1/playout/channels/{channelId}/output-arm	Arm or inhibit transmission outputs
GET/ POST	/api/v1/playout/channels/{channelId}/breaks	Read/create breaks
PATCH	/api/v1/playout/channels/{channelId}/breaks/{breakId}	Live/upcoming break edit
GET	/api/v1/playout/channels/{channelId}/asrun	As-run records

TAKE

```
POST /api/v1/playout/channels/KTX-PLAYOUT-01/commands
{
  "requestId":"take-001",
  "action":"take",
  "itemId":"7a2e4d9e-...",
  "transition":"cut"
}
```

Example response

```
{"ok":true,"code":"OK","data":{"onAirItemId":"7a2e4d9e-..."}}
```

Join-in-progress

```
POST /api/v1/playout/channels/KTX-PLAYOUT-01/commands
{
  "requestId":"jip-001",
  "action":"jip",
  "itemId":"live-election",
  "offsetSeconds":123.4
}
```

Example response

```
{"ok":true,"data":{"state":"PLAYING","joinedAtSeconds":123.4}}
```

Extend live event

```
POST /api/v1/playout/channels/KTX-PLAYOUT-01/commands
{"requestId":"extend-1","action":"extend","seconds":30,"reason":"Interview overrun"}
```

Example response

```
{"ok":true,"data":{"newPlannedEnd":"2026-09-17T15:40:30Z"}}
```

8. Channel Controller API

Method	Endpoint	Purpose
GET	/api/v1/channel-controller/channels	All managed channel clients
POST	/api/v1/channel-controller/channels	Register/add channel
GET	/api/v1/channel-controller/channels/{id}	Channel capabilities/health
PATCH	/api/v1/channel-controller/channels/{id}	Update name, endpoint, role, group
DELETE	/api/v1/channel-controller/channels/{id}	Remove managed channel
POST	/api/v1/channel-controller/groups	Create channel group
POST	/api/v1/channel-controller/commands	Send command to one/many channels
GET	/api/v1/channel-controller/multiview	Aggregated confidence/status model

Register channel

```
POST /api/v1/channel-controller/channels
{
  "channelId": "KTX-PLAYOUT-02",
  "name": "Sports HD",
  "host": "10.20.0.22",
  "apiPort": 9080,
  "group": "MAIN"
}
```

Example response

```
{"ok": true, "data": {"channelId": "KTX-PLAYOUT-02", "state": "CONNECTED"}}
```

Multi-channel command

```
POST /api/v1/channel-controller/commands
{
  "requestId": "group-hold-1",
  "targets": ["KTX-PLAYOUT-01", "KTX-PLAYOUT-02"],
  "action": "hold",
  "params": {"reason": "network maintenance"}
}
```

Example response

```
{"ok":true,"data":{"accepted":2,"failed":0}}
```

Query channel capability

```
GET /api/v1/channel-controller/channels/KTX-PLAYOUT-01
```

Example response

```
{
  "channelId": "KTX-PLAYOUT-01",
  "connected": true,
  "capabilities": ["NDI", "DeckLink", "ST2110", "SCTE35", "CEA708", "CG"],
  "outputArmed": true
}
```

9. CG Controller API

Method	Endpoint	Purpose
GET	/api/v1/cg-controller/targets	Available playout/channel targets
GET	/api/v1/cg-controller/channels/{id}/stack	Program graphics stack
POST	/api/v1/cg-controller/play	Play template/composition to one/many targets
POST	/api/v1/cg-controller/update	Update one/many on-air graphics
POST	/api/v1/cg-controller/take	Preview-to-Program take
POST	/api/v1/cg-controller/stop	Stop selected graphic
POST	/api/v1/cg-controller/clear	Clear selected/all layers

Play same graphic on multiple channels

```
POST /api/v1/cg-controller/play
{
  "requestId":"cg-multi-1",
  "targets":["KTX-PLAYOUT-01","KTX-PLAYOUT-02"],
  "bus":"PROGRAM",
  "template":"LOWER_THIRD_BASIC",
  "layer":20,
  "data":{"name":"Aarav Sharma","title":"Correspondent"}
}
```

Example response

```
{"ok":true,"data":{"played":["KTX-PLAYOUT-01","KTX-PLAYOUT-02"]}}
```

Different data per channel

```
POST /api/v1/cg-controller/play
{
  "requestId":"cg-per-channel-1",
  "targets":[
    {"channelId":"KTX-PLAYOUT-01","data":{"headline":"National"}},
    {"channelId":"KTX-PLAYOUT-02","data":{"headline":"Sports"}}
  ],
  "template":"HEADLINE_STRAP",
  "layer":30
}
```

Example response

```
{"ok":true,"data":{"accepted":2}}
```

Preview then TAKE

```
POST /api/v1/cg-controller/play
{"targets":["KTX-PLAYOUT-01"],"bus":"PREVIEW","template":"WEATHER","layer":40,"data":{"city":"Kathmandu","temp":"23"}}

POST /api/v1/cg-controller/take
{"targets":["KTX-PLAYOUT-01"],"layer":40}
```

Example response

```
{"ok":true,"data":{"bus":"PROGRAM","layer":40}}
```

10. CG Editor / Engine API

Method	Endpoint	Purpose
GET	/api/v1/cg/templates	Reusable template catalogue
POST	/api/v1/cg/templates	Create template
GET	/api/v1/cg/projects	Composition catalogue
GET	/api/v1/cg/projects/{id}	Complete composition graph/timeline
POST	/api/v1/cg/render	Render/preview with supplied data
POST	/api/v1/cg/projects/{id}/validate	Validate fonts/assets/bindings
POST	/api/v1/cg/channels/{channelId}/play	Direct engine play
POST	/api/v1/cg/channels/{channelId}/update	Live data update
POST	/api/v1/cg/channels/{channelId}/stop	Stop graphic

Render preview

```
POST /api/v1/cg/render
{
  "template":"LOWER_THIRD_BASIC",
  "resolution":"1920x1080",
  "data":{"name":"Test Name","title":"Reporter"}}
}
```

Example response

```
{"ok":true,"data":{"previewUrl":"/api/v1/cg/previews/5d9....png"}}
```

Live update

```
POST /api/v1/cg/channels/KTX-PLAYOUT-01/update
{
  "layer":20,
  "instanceId":"cg-inst-22",
  "data":{"title":"Senior Correspondent"}
}
```

Example response

```
{"ok":true,"data":{"instanceId":"cg-inst-22","updated":true}}
```

11. MAM API

Method	Endpoint	Purpose
GET	/api/v1/mam/assets	Search/list assets
POST	/api/v1/mam/assets	Register/import asset
GET/ PATCH	/api/v1/mam/assets/{assetId}	Read/update asset
POST	/api/v1/mam/assets/{assetId}/ checksum	Verify SHA-256
POST	/api/v1/mam/assets/{assetId}/proxy	Generate proxy
GET	/api/v1/mam/assets/{assetId}/versions	Versions
GET	/api/v1/mam/assets/{assetId}/ components	Video/audio/caption/data components
POST	/api/v1/mam/assets/{assetId}/archive	Archive
POST	/api/v1/mam/assets/{assetId}/restore	Restore
GET	/api/v1/mam/search	Advanced full-text/metadata search

Register asset

```
POST /api/v1/mam/assets
{
  "title": "News Package 17",
  "masterPath": "\\media\\online\\news17.mxf",
  "workflowState": "INGESTED",
  "tags": ["news", "package"],
  "rights":
  {
    "startUtc": "2026-09-01T00:00:00Z", "endUtc": "2027-08-31T23:59:59Z"
  }
}
```

Example response

```
{
  "ok": true,
  "data": {
    "assetId": "1a6aa0fb-...",
    "workflowState": "INGESTED"
  }
}
```

Archive

```
POST /api/v1/mam/assets/1a6aa0fb-.../archive
{"requestId": "archive-17", "destination": "nearline", "verifyChecksum": true}
```

Example response

```
{
  "ok": true,
  "data": {
    "taskId": "mam-task-1009",
    "state": "QUEUED"
  }
}
```

12. Ingest API

Method	Endpoint	Purpose
GET	/api/v1/ingest/sources	Discover/configured sources
GET/POST	/api/v1/ingest/jobs	List/create scheduled jobs
GET/PATCH	/api/v1/ingest/jobs/{jobId}	Read/update job
POST	/api/v1/ingest/jobs/{jobId}/start	Start immediately
POST	/api/v1/ingest/jobs/{jobId}/stop	Stop
GET	/api/v1/ingest/jobs/{jobId}/status	Progress and health

Scheduled master + proxy

```
POST /api/v1/ingest/jobs
{
  "name": "Studio 1 News Record",
  "sourceId": "decklink:0",
  "startUtc": "2026-09-18T12:00:00Z",
  "durationSeconds": 3600,
  "recurrence": "weekdays",
  "profiles": [
    { "role": "master", "preset": "XDCAM-HD422-50M-MXF" },
    { "role": "proxy", "preset": "H264-PROXY-MP4" }
  ],
  "segmentMinutes": 30,
  "registerToMam": true
}
```

Example response

```
{"ok": true, "data": { "jobId": "ing-301", "state": "SCHEDULED" }}
```

Start now

```
POST /api/v1/ingest/jobs/ing-301/start
{"requestId": "manual-start-1"}
```

Example response

```
{"ok": true, "data": {
  "state": "RECORDING", "masterPath": "...", "proxyPath": "..."
}}
```

13. NRCS and MOS API

Method	Endpoint	Purpose
GET/ POST	/api/v1/nrcs/rundowns	List/create rundown
GET/ POST	/api/v1/nrcs/rundowns/{id}/stories	List/create stories
PATCH	/api/v1/nrcs/stories/{storyId}	Update story/body/presenter/ duration
POST	/api/v1/nrcs/stories/{storyId}/mos-objects	Attach MAM/CG MOS object
POST	/api/v1/nrcs/rundowns/{id}/publish	Publish to Kashtrix Playout/MOS
GET	/api/v1/mos/status	MOS connection/status
POST	/api/v1/mos/send	Send supported MOS transaction

Attach CG MOS object

```
POST /api/v1/nrcs/stories/story-44/mos-objects
{
  "type": "CG",
  "template": "LOWER_THIRD_BASIC",
  "data": {"name": "Presenter", "title": "Anchor"},
  "targetLayer": 20
}
```

Example response

```
{"ok": true, "data": {"mosObjectId": "mos-cg-883"}}
```

Publish rundown

```
POST /api/v1/nrcs/rundowns/ro-20260917-evening/publish
{"targets": ["KTX-PLAYOUT-01"], "publishMos": true}
```

Example response

```
{"ok": true, "data": {"stories": 18, "mosObjects": 26, "playoutItems": 31}}
```

14. Prompter API

Method	Endpoint	Purpose
GET	/api/v1/prompter/{rundownId}	Rundown and current story
GET	/api/v1/prompter/{rundownId}/stories	Prompter story sequence
POST	/api/v1/prompter/{rundownId}/commands	start/stop/next/previous/goto/speed/font
GET	/api/v1/prompter/{rundownId}/state	Live state

Start scrolling

```
POST /api/v1/prompter/ro-20260917-evening/commands
{"action":"start","storyId":"story-44","speed":1.15}
```

Example response

```
{"ok":true,"data":{"running":true,"storyId":"story-44","speed":1.15}}
```

Next story

```
POST /api/v1/prompter/ro-20260917-evening/commands
{"action":"next"}
```

Example response

```
{"ok":true,"data":{"storyId":"story-45"}}
```

15. Traffic, As-Run, Monitoring and HA

Method	Endpoint	Purpose
POST	/api/v1/traffic/import	Import traffic/BXF-adapter payload
GET	/api/v1/traffic/events	Planned traffic events
POST	/api/v1/traffic/reconcile	Planned vs aired reconciliation
GET	/api/v1/asrun	Search/export as-run
GET	/api/v1/monitoring/alarms	Alarms
GET	/api/v1/monitoring/metrics	Metrics
GET	/api/v1/ha/nodes	HA nodes
POST	/api/v1/ha/failover	Controlled failover
POST	/api/v1/ha/nodes/{nodeId}/maintenance	Maintenance mode

Reconcile traffic

```
POST /api/v1/traffic/reconcile
{"channelId": "KTX-PLAYOUT-01", "fromUtc": "2026-09-17T00:00:00Z", "toUtc": "2026-09-18T00:00:00Z"}
```

Example response

```
{"ok": true, "data": {"planned": 244, "aired": 242, "matched": 238, "exceptions": 6}}
```

Controlled HA failover

```
POST /api/v1/ha/failover
{"requestId": "ha-33", "channelId": "KTX-PLAYOUT-01", "targetNode": "KTX-NODE-B", "reason": "maintenance"}
```

Example response

```
{"ok": true, "data": {"activeNode": "KTX-NODE-B", "outputArmed": true}}
```

16. MCP - Model Context Protocol integration

Specification target: MCP 2026-07-28. The remote Kashtrix MCP endpoint uses Streamable HTTP. The current specification moved to a stateless protocol core, requires HTTP routing headers such as `Mcp-Method` and `Mcp-Name`, and moved asynchronous Tasks into the `io.modelcontextprotocol/tasks` extension. Legacy HTTP+SSE is not the target for new Kashtrix deployments.

Endpoint: `POST /mcp` over HTTPS. MCP does not bypass broadcast safety. Each tool maps to the same Kashtrix service and permission checks used by REST/TCP/WebSocket.

MCP capability mapping

MCP type	Kashtrix examples	Purpose
Tools	playout.take, playout.hold, mam.search, mam.archive, ingest.schedule, cg.play, nracs.publish	Actions with explicit schemas
Resources	kashtrix://channels/{id}/status, kashtrix://mam/assets/{id}, kashtrix://nracs/rundowns/{id}	Read-only contextual data
Prompts	rundown_qc_review, ingest_failure_triage, daily_asrun_summary	Reusable operator/engineering workflows
Tasks extension	mam.archive, mam.restore, ingest.long_record, proxy.generate, reconciliation.run	Long-running operations with task handles

Recommended MCP tools

Tool	Purpose
playout_get_status	Read channel state; no mutation.
playout_take	TAKE selected/specified item after RBAC and output safety validation.
playout_hold	Hold/unhold channel.
playout_extend	Extend current event.
channel_controller_list	List channel clients and readiness.
channel_controller_command	Dispatch command to selected managed channels.
cg_controller_play	Play same/different data across selected channels.
cg_controller_update	Update on-air graphic instances.
mam_search_assets	Search MAM by text/metadata/rights/workflow.
mam_get_asset	Read asset components, versions, QC, storage.
mam_generate_proxy	Start proxy task.
mam_archive_asset	Start archive task.
ingest_list_sources	Read available ingest sources.
ingest_schedule_job	Create scheduled master/proxy ingest.
ingest_start_job	Start permitted job.
nrcs_get_rundown	Read rundown/stories/MOS objects.
nrcs_publish_rundown	Publish validated rundown.
prompter_get_state	Read active story/state.
traffic_reconcile	Start planned-vs-aired reconciliation task.
monitoring_get_alarms	Read active alarms.
ha_get_state	Read HA state; mutation should require engineer/admin.

MCP tool call example - MAM search

```
POST /mcp
Mcp-Method: tools/call
Mcp-Name: mam_search_assets
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": 12,
  "method": "tools/call",
  "params": {
    "name": "mam_search_assets",
    "arguments": {
      "query": "election
package", "workflowState": "READY", "limit": 10
    }
  }
}
```

MCP long-running Task example - archive

```
{
  "jsonrpc": "2.0",
  "id": 13,
  "method": "tools/call",
  "params": {
    "name": "mam_archive_asset",
    "arguments": {
      "assetId": "1a6aa0fb-...", "destination": "nearline", "verifyChecksum": true,
      "_meta": { "extensions": ["io.modelcontextprotocol/tasks"] }
    }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 13,
  "result": {
    "resultType": "task",
    "task": { "taskId": "mcp-task-x8F...", "status": "working" }
  }
}
```

Client polls tasks/get (or listens through the Tasks extension subscription mechanism where supported) until the archive/restore/proxy/reconciliation task completes. Task IDs must be unguessable and scoped to the authorized caller.

17. Errors, idempotency and audit

Code	Meaning	Typical response
INVALID_REQUEST	Schema/parameter failure	400
UNAUTHENTICATED	Missing/invalid credentials	401
FORBIDDEN	Identity lacks role	403
NOT_FOUND	Asset/channel/job/object missing	404
CONFLICT	State/lock/duplicate conflict	409
COMMAND_REJECTED	Valid request cannot execute in current state	422
RATE_LIMITED	Client rate exceeded	429
MODULE_UNAVAILABLE	Module not ready	503
HARDWARE_UNAVAILABLE	Required device/adapter unavailable	503

```
{
  "ok": false,
  "code": "COMMAND_REJECTED",
  "message": "Program output is inhibited on standby node",
  "traceId": "01J...",
  "details": {"nodeRole": "STANDBY", "outputArmed": false}
}
```

Appendix A. Complete endpoint catalogue

Module	Method	Endpoint	Purpose
Platform	GET	/api/v1/health	System/module health
Platform	GET	/api/v1/capabilities	Implemented/adaptor-required capabilities
Platform	WS	/api/v1/events/ws	Realtime events + commands
Playout	GET	/api/v1/playout/channels/{id}/status	Channel status
Playout	POST	/api/v1/playout/channels/{id}/commands	Transport/automation command
Playout	POST	/api/v1/playout/channels/{id}/output-arm	Output interlock
Channel Controller	GET	/api/v1/channel-controller/channels	Managed clients
Channel Controller	POST	/api/v1/channel-controller/commands	Multi-channel command
CG Controller	POST	/api/v1/cg-controller/play	Multi-target graphic play
CG Controller	POST	/api/v1/cg-controller/update	On-air update
CG Controller	POST	/api/v1/cg-controller/take	Preview to Program
CG	GET	/api/v1/cg/templates	Templates
CG	POST	/api/v1/cg/render	Preview render
MAM	GET/ POST	/api/v1/mam/assets	Search/register assets
MAM	POST	/api/v1/mam/assets/{id}/proxy	Proxy task
MAM	POST	/api/v1/mam/assets/{id}/archive	Archive task
Ingest	GET/ POST	/api/v1/ingest/jobs	Schedule/list ingest
Ingest	POST	/api/v1/ingest/jobs/{id}/start	Start ingest
NRCS	GET/ POST	/api/v1/nrcs/rundowns	Rundowns
NRCS	POST	/api/v1/nrcs/rundowns/{id}/publish	Publish
Prompter	POST	/api/v1/prompter/{id}/commands	Prompter control

Module	Method	Endpoint	Purpose
Traffic	POST	/api/v1/traffic/reconcile	Reconciliation
Monitoring	GET	/api/v1/monitoring/alarms	Alarms
HA	POST	/api/v1/ha/failover	Controlled failover
MCP	POST	/mcp	MCP Streamable HTTP

Appendix B. Realtime event catalogue

Event	Meaning
playout.item.started	Primary item entered Program
playout.item.completed	Primary item completed
playout.state.changed	Play/pause/stop/hold changed
playout.next.changed	Next editorial item changed
playout.output.armed	Output arm/inhibit changed
playout.break.changed	Break structure changed
channelController.channel.connected	Managed channel connected
channelController.channel.disconnected	Managed channel disconnected
cg.played	Graphic started
cg.updated	On-air data changed
cg.stopped	Graphic stopped
ingest.job.started	Recording started
ingest.job.progress	Recording progress
ingest.job.completed	Recording/post-actions complete
mam.asset.registered	Asset registered
mam.asset.workflow.changed	Workflow/QC/archive changed
nracs.rundown.published	Rundown published
nracs.story.changed	Story/MOS assignment changed
prompter.story.changed	Prompter story changed
traffic.reconciliation.completed	Reconciliation complete
alarm.raised	Alarm raised
alarm.cleared	Alarm cleared
ha.active.changed	Active HA node changed

Appendix C. API acceptance checklist

- REST/WebSocket/TCP commands invoke the same validated service handler.

- Channel Controller multi-target command returns per-channel result, not one fake global success.
- CG Controller Preview and Program remain independent and multi-channel targeting is verified.
- TCP heartbeat, reconnect, malformed JSON and oversized message handling are tested.
- WebSocket reconnect/resubscribe and event sequence gaps are tested.
- UDP loss/duplicate/reorder behavior is tested; critical commands do not depend on unacknowledged UDP.
- API key/JWT/OIDC mapping to RBAC is tested.
- Idempotency prevents duplicate TAKE/archive/schedule commands.
- MCP tools cannot bypass roles, output interlocks or hardware readiness.
- MCP long operations return real task state and final result/error.
- Audit contains transport, identity, request ID, module, target and outcome.

References used for the MCP section

Model Context Protocol official 2026-07-28 release documentation and official Tasks extension documentation were used to align the MCP chapter. The Kashtrix REST/TCP/UDP/WebSocket APIs in this manual are Kashtrix product contracts, not claims that they are part of MCP.

- Model Context Protocol - 2026-07-28 specification release (official project blog).
- MCP Tasks extension - 2026-07-28 specification.
- Official MCP TypeScript SDK documentation for Streamable HTTP, tools, resources and transports.